

C Programming

5_arrays

Average score?

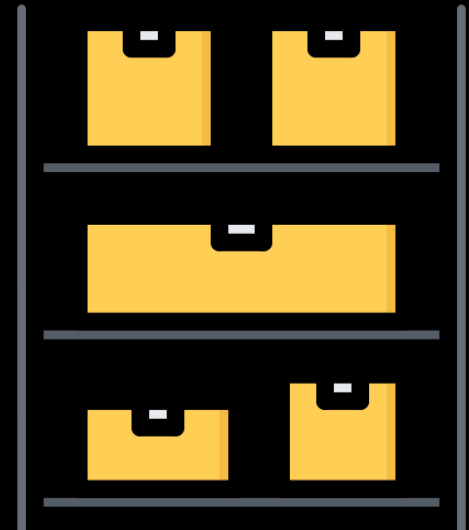
	score1	score2	score3	
	77	43	100	
	Паша	Маша	Даша	

Arrays

Массивы используются для хранения нескольких значений в одной переменной вместо объявления отдельных переменных для каждого значения.

Если переменная - это коробка со значением, то массив - это шкаф с этими коробками

Массивы во многих ЯП (в том числе Си) умеют хранить значения **только одного** типа данных, т.е. один массив не может хранить сразу и **целые** и **вещественные** типы данных



Arrays

```
int scores[3] = {77, 43, 100};
```

	[0]	[1]	[2]	
Value	77	43	100	
Address	200	204	208	

Массивы в C/C++, Python, Java, Go и многих других ЯП индексируются с 0
Исключения, где массивы начинаются с индекса 1: MATLAB, lua, R, Julia..

```
printf("Scores: ");  
for (int i = 0; i < 3; i++) {  
    printf("%d, ", scores[i]);  
}  
printf("\n");
```

```
#define N 5
```

```
int a[N];
```

```
int b[5] = {1, 2, 3, 4, 5};
```

```
float c[] = {1.5, 3.2, 2.5};
```

```
int d[6] = {[3] = 9};
```

```
char e[N] = {0};
```

```
int a[5] = {5, 6, 7, 8, 9};
```

```
int b[5] = {1, 2, 3, 4, 5};
```

```
b = a; // Error
```

```
a = {5, 3}; // Error
```

```
a[2] = -17; // Ok
```

```
a[0] = b[4]; // Ok
```

```
a[2.5] = 33; // Error
```

```
b[4000] = 100; // Error, segfault
```

```
a[-1] = 8; // Ok, possible segfault
```

Segmentation fault - ошибка во время выполнения программы, когда программа попыталась обратиться к памяти, к которой доступ не разрешен.

array size

```
int m[] = {1, 2, 3}; // 3 элемента  
int array_size = sizeof(m) / sizeof(m[0]); // 12/4=3
```


Delete/insert

В "обычном" массиве нельзя удалить элемент и уменьшить/увеличить размер самого массива. Такое можно будет делать с **динамическими массивами**, скоро их посмотрим

```
void print_arr(int n, int array[n]) {  
    for (int i = 0; i < n; i++) {  
        printf("array[%d] = %d\n", i, array[i]);  
    }  
    printf("\n");  
}
```

```
int scores[N] = {77, 43, 100, 55, 11};  
print_arr(N, scores);
```

```
int array[M] = {1, 2, 3};  
print_arr(M, array);
```

Random numbers

```
#include <stdlib.h>
// ...
int random = rand();
```

rand() - ф-ия, что возвращает случайное **целое** число (целое в диапазоне от 0 до RAND_MAX, где RAND_MAX - определен в stdlib.h)

Random numbers

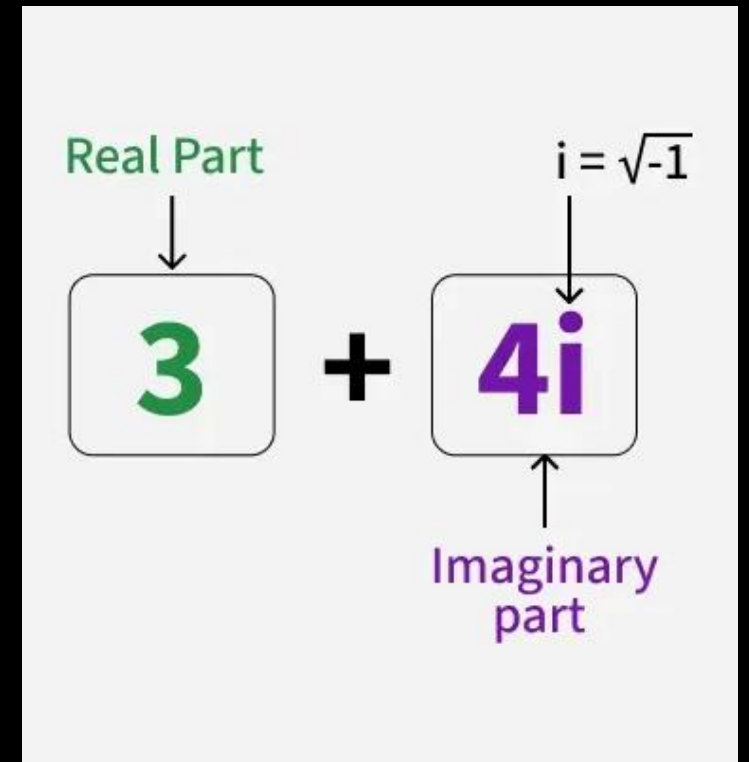
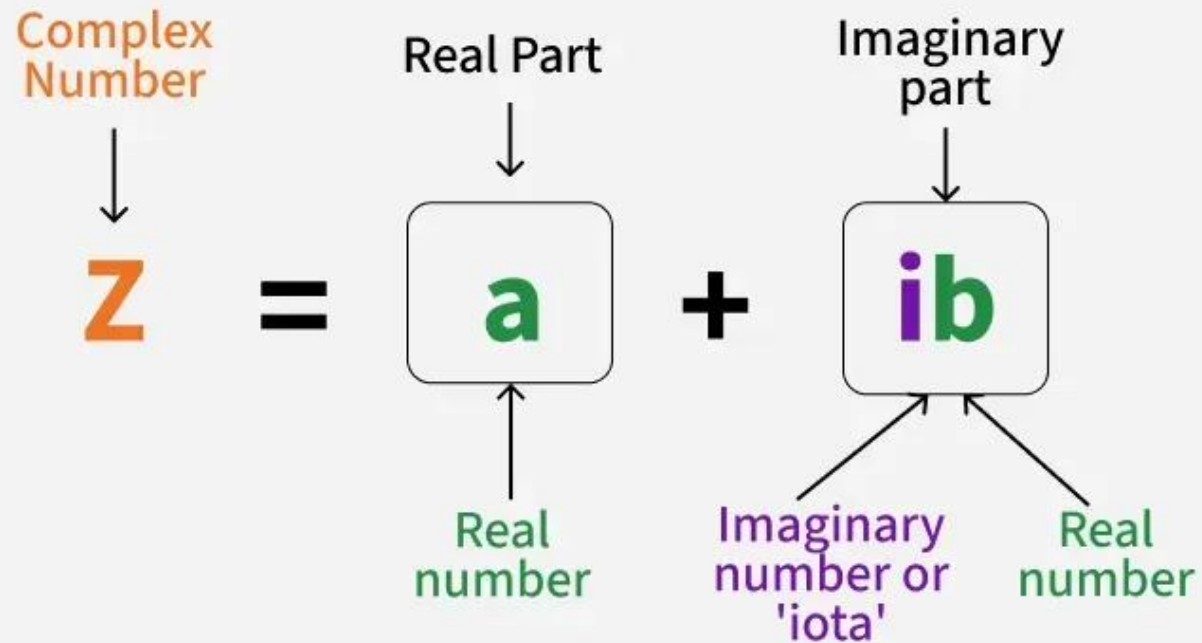
- **srand()** - ф-ия задания seed (семя) для псевдорандома. Seed - некоторое число, что каким-то образом влияет на формулу случайных чисел
- **time()** - ф-ия возвращающая кол-во секунд прошедших с 00:00:00 UTC, 1 Января, 1970 года, называется - Unix timestamp (timestamp - временная метка).

<https://www.unixtimestamp.com/>

```
srand(time(NULL)); // достаточно сделать 1 раз  
printf("Unix timestamp = %ld\n", time(NULL));  
printf("Random [0..99] = %d\n", rand() % 100);
```

Самостоятельно: заполнить массив размером N случайными числами от 0 до 999 и найти в нем минимальный элемент - вывести его на экран

Complex Numbers



Комплексное число – выражение вида $a + bi$, где a и b некоторые целые либо вещественные числа, а i это корень из -1 .

Как представить комплексное число и положить его в компьютер?

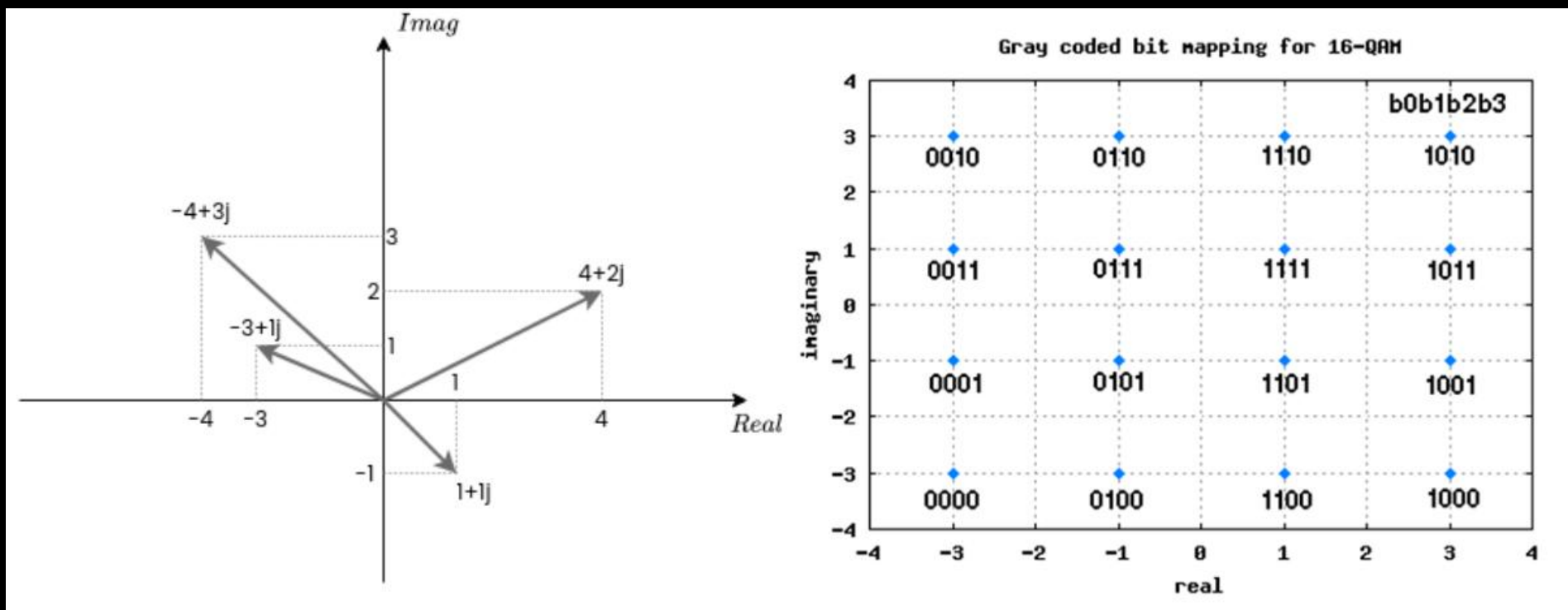
Одно **комплексное число** создают из двух чисел — a и b , а i пропускается. Соответственно массив из 3 комплексных чисел есть массив из 6 обычных чисел. И тогда числа под четными индексами — это **реальная часть** комплексного числа, а нечетные индексы **мнимая часть**:

[RE0, IM0, RE1, IM1, RE2, IM2],

- RE — Real (реальная часть),
- IM — Imag (imaginary, мнимая часть).

Так, например, наши с вами телефоны из воздуха достают большие массивы **комплексных чисел**, когда работают с беспроводными сетями типа **Wi-Fi** или **4G**, **5G**..

Эти комплексные числа образуют различные "**созвездия**", которые затем преобразуются в более понятные данные из `0` и `1`



Допустим нам из воздуха пришло 10 комплексных целых чисел, где реальная и мнимая части занимают по 16 бит, как мы можем упаковать их в массив?

```
short int complex[20];
```

- `complex[четный_индекс]` - реальная часть
- `complex[нечетный_индекс]` - мнимая часть

```
int complex[10];
```

- `complex[i]` - комплексное число
 - `(complex[i] & 0xffff)` - реальная часть
 - `(complex[i] >> 16) & 0xffff` - мнимая часть

Time to sort

TODO

Github

<https://github.com/kruffka/C-Programming>

